

N8N Automations

A Deep Dive into 59 Workflow Automations

Exhaustive walkthrough, assuming no prior knowledge

What you are about to read

This project is not a program in the usual sense. There is no source file you can run, no test suite, no build. It is a collection of 59 *automations*: business chores, each one wired together as a flowchart inside a tool called n8n, then exported to a file so it can be read, reviewed and re-imported.

This document explains what n8n is, what an exported workflow actually contains, what each family of automations does and why, and where the collection is weak. Everything here was checked against the exported files themselves, not taken on trust from the project's own documentation.

Contents

1	What this project is, and what it is not	3
1.1	The one-sentence version	3
1.2	What it is not	3
2	n8n, explained from zero	3
2.1	The problem automation tools solve	3
2.2	The five words you need	4
2.3	The shape of a workflow	5
2.4	Four more terms this repository leans on heavily	5
3	The exported file: what is actually in it	6
3.1	The top level	6
3.2	One node, dissected	6
3.3	How a wire is stored, and why the shape is weird	7
4	Repository anatomy	8
4.1	The layout	9
4.2	The four-file convention	9
5	The verified census	10
5.1	What the collection is built out of	10
5.2	How these automations start	11
6	A taxonomy of the 59	12
6.1	Family 1: Lead generation and outreach (11)	12
6.2	Family 2: Content generation (14)	12
6.3	Family 3: Video production (7)	13
6.4	Family 4: SEO (5)	13
6.5	Family 5: Assistants and chatbots (9)	13
6.6	Family 6: Business operations (6)	13
6.7	Family 7: Utilities (7)	13

7	Walkthrough 1: CRM lead to Stripe invoice	13
7.1	What it does	14
7.2	Why three raw HTTP calls sit next to a native Stripe node	14
7.3	The flaws, verified	14
8	Walkthrough 2: the Ultimate Personal Assistant	15
8.1	What it does	15
8.2	Why this design is good	15
9	Walkthrough 3: the RAG ingestion pipeline	16
9.1	What it does	16
9.2	The interesting parts	17
10	Walkthrough 4: the cold email outreach system	17
10.1	What it does	17
10.2	The five-attempt pattern	18
11	Cross-cutting patterns	19
11.1	Google Sheets as the database	19
11.2	Polling for slow jobs	20
11.3	Trusting the LLM's output shape	20
11.4	Sticky notes as documentation	21
11.5	Orphaned nodes: the visual tool's characteristic debt	21
12	Security posture	22
12.1	How n8n stores credentials, and why that saves this project	22
12.2	The audit, and its result	23
13	The honest summary	23
13.1	What is genuinely good	24
13.2	What is weak, ordered by what it would actually cost	24
13.3	The maturity question, stated plainly	24
13.4	What was inferred rather than confirmed	25
14	What an interviewer will ask	25

1 What this project is, and what it is not

1.1 The one-sentence version

Fifty-nine business processes, each built as a flowchart that then runs itself, exported as text files and documented honestly, faults included.

1.2 What it is not

It is worth being precise about this early, because the shape of the project is unusual and it is easy to misjudge in both directions.

- **It is not a codebase.** There is no application to compile. The only executable-ish artifacts are JavaScript snippets embedded inside individual workflow steps, and one small Python script used to draw pictures of the workflows for the website.
- **It is not a library or a product.** Nothing here is meant to be imported by another program. Each automation is a self-contained job.
- **It is not a live system.** These are *exports*, snapshots taken out of a running n8n instance. Importing one into a fresh n8n gives you the shape of the automation but none of its account access, which has to be supplied again by hand.
- **It is not a set of toy examples.** Most of these ran, or were built to run, against real business accounts at a real agency: a real payment processor, a real customer database, real inboxes.

But the "runs itself" claim needs an immediate qualifier

Of the 80 exported workflow files in this repository, exactly **9** carry `active: true`. The other 71 are marked inactive. In n8n, an inactive workflow's automatic triggers do not fire; it only runs when a human presses the button in the editor.

That does not mean 71 automations are broken. An export is a snapshot, and a workflow is routinely deactivated before export, or exported from a development copy. But it does mean this repository is **not evidence that 59 automations are running in production right now**, and nobody should describe it that way. It is evidence that 59 automations were designed and built. Section 13 returns to this.

2 n8n, explained from zero

Nothing in this section assumes you have seen n8n, or any automation tool. If you already know n8n well, skip to Section 3.

2.1 The problem automation tools solve

Consider a real chore: *when a sales deal is marked "won" in our customer database, create a customer record in our payment processor and send them an invoice.*

You could do that by hand, every time, in about four minutes. You could also write a program: talk to the database's API, talk to the payment processor's API, handle the errors, host it somewhere, keep it alive. That is maybe a day of work and a permanent maintenance burden, for four minutes of chore.

Jargon: API

Short for Application Programming Interface. A door that a piece of software opens deliberately so that *other* software can talk to it, as opposed to the screens it opens for humans.

When this document says a workflow "calls Stripe's API", it means the workflow sends Stripe a structured message over the internet asking it to do something, and Stripe sends a structured answer back.

n8n sits between those two options. It is a **visual workflow automation tool**: you drag boxes onto a canvas, wire them together with lines, and the tool runs the resulting flowchart for you. It already knows how to talk to hundreds of services, so "talk to the payment processor's API" becomes "drag the Stripe box onto the canvas".

The honest framing of what this skill is

Dragging boxes sounds like it is not engineering. Building one of these badly is easy and building one of these well is not, and the reason is that everything hard about software is still present: state, failure, retries, idempotency, data shape mismatches, rate limits, credentials, and the question of what happens on the run where the third API call times out after the first two already took effect.

The visual editor removes the typing. It does not remove the thinking. Most of Section 13 is a catalogue of places in this very repository where the thinking was not done, and each one is a bug that would be equally possible in handwritten code.

2.2 The five words you need

Jargon: Node

One box on the canvas: a single step. "Send a Gmail message" is a node. "Ask an AI model for text" is a node. "If the amount is over 500, go left, otherwise go right" is a node. A node takes data in, does one thing, and passes data out. Across this repository there are **1,645** nodes.

Jargon: Connection

A line between two nodes, saying "when this one finishes, run that one next, and hand it your output". Connections are directional. The set of all connections is the flowchart.

Jargon: Trigger

The special node that *starts* a workflow. Every other node runs because something before it ran; a trigger runs because something happened in the outside world. A schedule trigger fires at 8am. A webhook trigger fires when another system sends this workflow a message. A Telegram trigger fires when someone texts the bot. A manual trigger fires when a human clicks "Execute" in the editor.

Jargon: Credential

A stored login for an outside service, kept in n8n's own encrypted credential store and referenced by nodes rather than typed into them. This distinction matters enormously for this repository and Section 12 is devoted to it.

Jargon: Expression

A small piece of live text inside a node's settings that gets computed at run time instead of being fixed. Written in double curly braces. If a node's "To" field contains an expression referring to the email address from an earlier step, then each run sends to a different person.

A field without an expression is a hardcoded constant and sends to the same person forever, which, as Section 13 shows, is a mistake this repository makes more than once.

2.3 The shape of a workflow

Figure 1 shows the whole idea in one picture, using the deal-won example.

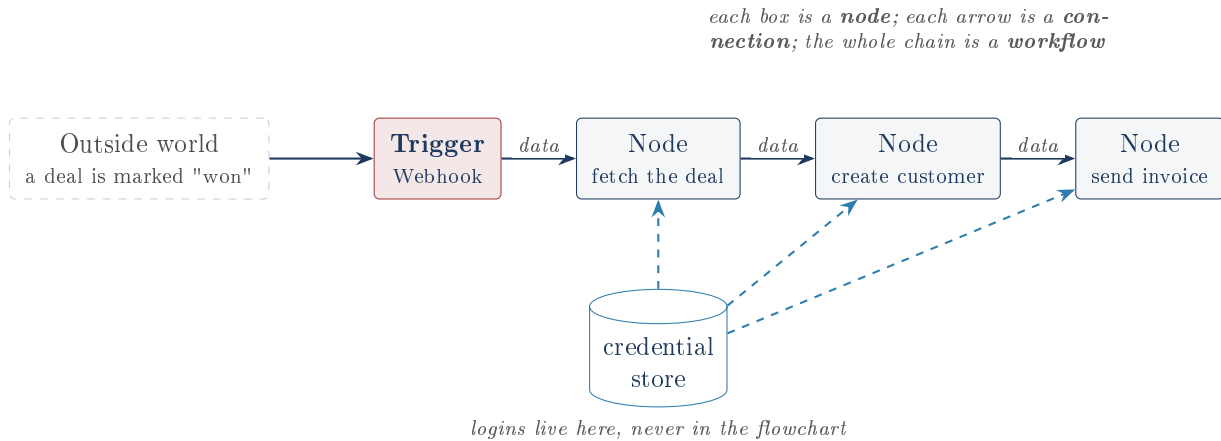


Figure 1: The vocabulary in one picture. A trigger starts the run; nodes do one thing each; connections carry data forward; credentials are referenced from a separate store rather than embedded.

2.4 Four more terms this repository leans on heavily

Jargon: Sub-workflow

A workflow called by another workflow, the way a function is called by other code. The caller uses an `executeWorkflow` node; the callee starts with an `executeWorkflowTrigger` node instead of a normal trigger. This repository uses the pattern 17 times, and it is how the largest systems here are organized.

Jargon: AI Agent (and its tools)

A node that hands a question to a large language model *along with a list of things the model is allowed to do*, and then lets the model decide which of them to do. Those "things" are called tools, and each is another node plugged into the agent. An agent with a calendar tool and an email tool can be asked "move my 3pm and tell Sam", and it will work out that this means two tool calls in a particular order. There are **109** agent nodes in this repository.

Jargon: LLM (Large Language Model)

The kind of AI model behind ChatGPT and its competitors: it takes text and produces plausible continuation text. Crucially for this document, an LLM's output is *probabilistic*, not guaranteed. Ask for JSON and you usually get JSON. "Usually" is the entire source of a recurring class of bug in Section 13.

Jargon: RAG (Retrieval-Augmented Generation)

A technique for making an LLM answer from *your* documents instead of its general training. You chop the documents into chunks, convert each chunk into a list of numbers capturing its meaning (an *embedding*), and store them in a vector database. At question time you embed

the question, find the chunks whose numbers are nearest, and paste those chunks into the prompt. The model then answers from real text you supplied rather than from memory. Two systems here do this.

3 The exported file: what is actually in it

Every automation in this repository is, on disk, one or more `.json` files. This section explains exactly what is inside one, because everything else in this document is a reading of these files.

Jargon: JSON

A plain-text format for structured data, made of nested lists and key/value pairs. It is what most web services speak, and it is human-readable, which is what makes reviewing these exports possible at all.

3.1 The top level

The export is one JSON object. The two keys that matter are **nodes** (a flat list of every box on the canvas) and **connections** (a map describing every wire). The rest is metadata: identifiers, timestamps, the workflow's display name, and the **active** flag.

```

1  {
2    "name": "YT Transcript",
3    "active": false,
4    "nodes": [ ... 3 node objects ... ],
5    "connections": {
6      "Webhook": { "main": [ [ { "node": "HTTP Request", "index": 0 } ] ] },
7      "HTTP Request": { "main": [ [ { "node": "Respond to Webhook", "index": 0 } ] ] }
8    },
9    "settings": { "executionOrder": "v1" },
10   "pinData": {},
11   "tags": []
12 }

```

Listing 1: The whole of a small export, with values that identify a live endpoint replaced by placeholders. This is `youtube-transcript-fetcher`, the smallest automation here at 3 nodes.

Connections are keyed by node *name*, not by ID

Look closely at the `connections` map: the wires refer to nodes by their human-readable display name ("`HTTP Request`"), even though every node also carries a unique ID. The same is true of expressions inside nodes, which reach back to earlier steps by writing `$( 'Some Node Name' )`.

This is the single most important structural fact about n8n for understanding Section 13, because it means **renaming a node in the editor can silently break any expression that referred to it**. n8n fixes up the wires when you rename, but an expression is just text inside a field, and text does not get fixed up. This repository contains a workflow where exactly this happened, three times, in one file.

3.2 One node, dissected

```
1 {
2   "name": "Webhook",           <-- what connections and expressions call it
3   "type": "n8n-nodes-base.webhook", <-- which KIND of box this is
4   "typeVersion": 2,          <-- which generation of that box
```

```

5  "position": [0, 0],                <-- where it sits on the canvas (cosmetic)
6  "id": "<uuid>",                    <-- unique, but NOT what wires use
7  "webhookId": "<uuid>",
8  "parameters": {                  <-- everything the user configured
9    "httpMethod": "POST",
10   "path": "<uuid>",
11   "options": { "allowedOrigins": "*" }
12 }
13 }
```

Listing 2: A single node object. This is the webhook trigger from the same file; the real `path` and `webhookId` are the live URL of this endpoint, so they are shown here as placeholders rather than reproduced.

The `type` field is the interesting one. It is a namespaced string, and the namespace tells you where the node came from:

Namespace	Meaning	Count
n8n-nodes-base.*	Ships with n8n itself	1,298
@n8n/n8n-nodes-langchain.*	Official AI add-on pack	336
n8n-nodes-scrapegraphai.*	Third-party community pack	4
n8n-nodes-fathom.*	Third-party community pack	2
@apify/n8n-nodes-apify.*	Third-party community pack	1
@devlikeapro/n8n-nodes-waha.*	Third-party community pack	1
@elevenlabs/n8n-nodes-elevenlabs.*	Third-party community pack	1
n8n-nodes-templated.*	Third-party community pack	1
n8n-nodes-guuid-generator.*	Third-party community pack	1
		1,645

Seven community packages are an undocumented portability constraint

The bottom seven rows are not part of n8n. They are third-party packages that must be installed into the instance before the workflow using them will even *open*, let alone run. Eleven nodes across seven separate packages, and each package is a separate install and a separate maintenance risk: a community node abandoned by its author is a workflow that stops opening after an n8n upgrade.

Anyone importing `fathom-native-node-integration`, `whatsapp-ai-sales-lead-system`, `uuid-generator-utility` or any of the four others into a stock n8n gets an unrecognized-node error. Neither the repository root README nor those folders' setup sections leads with that fact. It is a small thing that will cost a first-time importer twenty confused minutes, and the fix is one line in each affected `CREDENTIALS.md` or setup section.

3.3 How a wire is stored, and why the shape is weird

```

1  "connections": {
2    "Loop Over Items": {           // the SOURCE node's name
3      "main": [                   // which KIND of output (see below)
4        [ {"node": "Switch2", "index": 0} ], // output slot 0 -> these targets
5        [ {"node": "Download File", "index": 0} ] // output slot 1 -> these targets
6      ]
7    }
8  }
```

Listing 3: The nesting of the connections map, explained.

Three levels of nesting, each meaning something:

1. **The connection kind** ("main"). Ordinary data flow is `main`. AI nodes use other kinds: `ai_languageModel`, `ai_tool`, `ai_memory`, `ai_embedding`. These are how a model, a tool or a memory gets attached to an agent.
2. **The output slot** (the outer array). A node with one output has one slot. An `If` node has two: slot 0 is "true", slot 1 is "false". A `Switch` can have many. A loop node uses slot 0 for "done" and slot 1 for "next batch".
3. **The target list** (the inner array). One output slot can feed several nodes at once, which is how the canvas fans out.

The AI connections run *backwards*, and this confuses everyone

For an agent, the `ai_tool` and `ai_languageModel` connections are stored on the *tool*, pointing *at* the agent. Reading the raw JSON, it looks like the Gmail tool feeds the agent, in the same direction as ordinary data flow. It does not: the agent calls the tool.

The arrow in the file describes attachment ("I am equipment belonging to that agent"), not data flow. Anyone reading these exports for the first time, or writing a script to analyse them, will get the direction wrong once. Figure 6 draws it the way it actually behaves rather than the way it is stored.

4 Repository anatomy

4.1 The layout

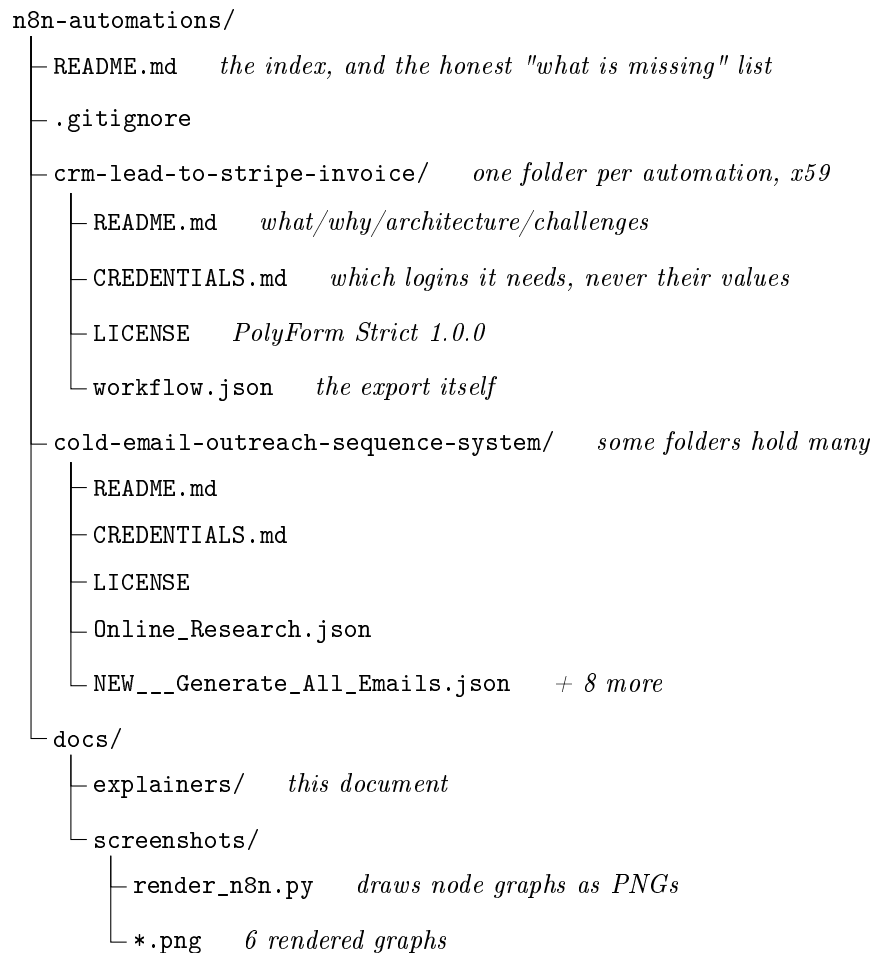


Figure 2: The repository is one flat tier of automation folders plus a `docs/` directory. There is no shared code, no top-level manifest, and no build.

4.2 The four-file convention

Every one of the 59 folders holds the same four things, and the consistency is real, not aspirational. Verified by inspection:

File	In all 59?	Role
README.md	yes	The prose. Nine standard headings.
CREDENTIALS.md	yes	The external accounts needed, by name and purpose.
LICENSE	yes	PolyForm Strict 1.0.0, byte-identical in 58 of 59.
*.json	yes	One or more exports. 80 files across 59 folders.

The README headings are near-perfectly uniform across all 59 folders: *What it is*, *Why it exists*, *Features*, *Architecture*, *Setup*, *Usage*, *Challenges*, *What I learned*, *What I'd do differently*. Two folders lack a *What it is* heading and one lacks *Why it exists* and *Features*, each having replaced it with something more specific such as *Naming note (read this first)*.

The 59th LICENSE differs, by one byte

`uuid-generator-utility/LICENSE` is identical to the other 58 except that it lacks a trailing newline. This is not a licensing problem and has no legal effect. It is worth a single sentence only because it is a tell: the files were placed by hand or by a script that did not normalize its output, rather than copied from one canonical source.

Jargon: PolyForm Strict 1.0.0

A source-available, not open-source, licence. Anyone may read the code; nobody may use it, modify it or redistribute it without a separate agreement. The sensible choice for a portfolio: the work can be examined by an interviewer but not lifted into a competitor's product.

5 The verified census

The project claims 59 automations. That claim is checkable and it was checked, by enumerating the tracked files and parsing every export rather than by counting the README's bullet points.

Measure	Count	How it was established
Automation folders	59	Tracked top-level directories, minus <code>docs/</code>
Exported workflow files	80	Every tracked <code>*.json</code>
Total nodes	1,645	Sum over every export's <code>nodes</code> array
Distinct node types	93	Unique <code>type</code> strings
Sticky notes (documentation)	186	<code>stickyNote</code> nodes
Functional nodes	1,459	Total minus sticky notes
Workflows marked active	9 of 80	<code>active: true</code>
Disabled nodes	45	<code>disabled: true</code>
Workflows with pinned test data	9	Non-empty <code>pinData</code>

The headline number is honest, with a caveat worth stating out loud

59 automations is correct, if "automation" means "one documented folder solving one business problem". It is the count of folders, and there are exactly 59.

The caveat is that 59 folders contain 80 workflow files, because six folders hold multi-workflow systems (the cold email system alone is 10 files and 275 nodes). So "59" undercounts the workflow files and overcounts the independent programs, depending on which you meant. It is a defensible number, not an inflated one, and the alternative counts are 80 files or 1,645 nodes.

A different claim on the portfolio website is not about this project

The website's project grid shows the metric "33 automation hooks" on a card. That card belongs to **reelflow-studio**, a different project entirely, and the number has nothing to do with this repository. This repository's card correctly reads "59 automations". Nothing here needs fixing; the number is noted only so it is not mistaken for a claim this project failed to substantiate.

5.1 What the collection is built out of

Node type frequency is a decent x-ray of a workflow collection's character.

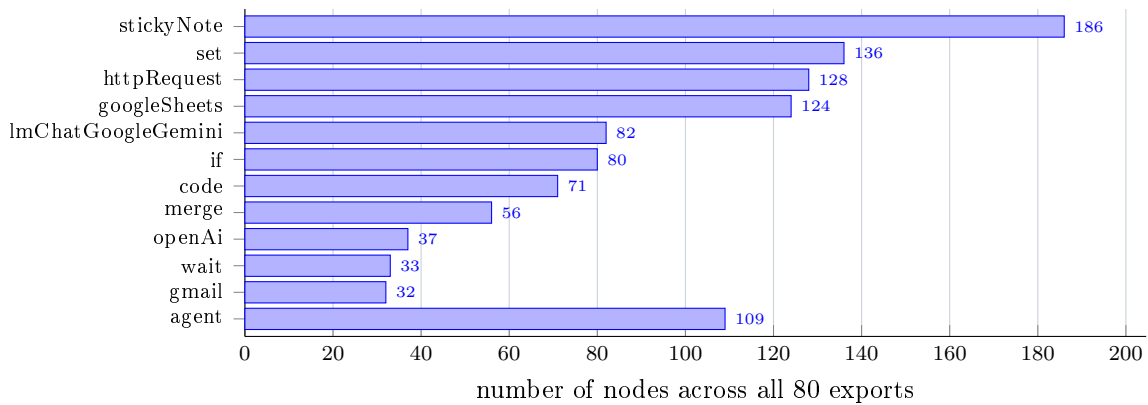


Figure 3: The twelve most common node types. Read it as a portrait: this is a collection about *moving data between web services and an LLM*, glued with spreadsheets.

Four things fall straight out of that chart:

- **Sticky notes are the single most common node** (186). A sticky note does nothing at run time; it is a comment on the canvas. Roughly one node in nine here exists purely to explain the others. That is a genuinely good sign about how these were built, and it is unusual.
- **Google Sheets is the database** (124 nodes, plus 5 Sheets triggers). Section 11 argues this is both the collection's cleverest habit and its biggest architectural weakness.
- **httpRequest appears 128 times**, meaning that for a large fraction of the services touched, no purpose-built n8n node existed and the author wrote the raw API call by hand. This is the "when the visual tool runs out, drop to the API" move, and it is used constantly.
- **AI is not a garnish here.** 109 agent nodes plus 82 Gemini models plus 37 OpenAI nodes plus 17 OpenAI chat models. A fifth of all nodes are AI-related.

5.2 How these automations start

Trigger type	n	Meaning
manualTrigger	28	A human clicks Execute
executeWorkflowTrigger	16	Called by another workflow
scheduleTrigger	14	Fires on a clock
webhook	14	Another system posts to a URL
formTrigger	11	Someone submits an n8n-hosted form
chatTrigger	7	Someone types into a chat box
telegramTrigger	5	Someone messages a Telegram bot
googleSheetsTrigger	5	A spreadsheet row changed
googleDriveTrigger	3	A file appeared or changed
errorTrigger	2	Another workflow failed
fathomTrigger	2	A meeting recording finished
calTrigger	1	A booking was made
wahaTrigger	1	A WhatsApp message arrived

28 manual triggers is the most revealing number in this document

manualTrigger is the most common trigger in the collection. A manual trigger is what n8n gives you while you are *building*: it exists so you can press Execute and watch data flow. A finished, deployed automation is normally started by a schedule, a webhook or an event, not by a person opening the editor.

Combined with only 9 of 80 workflows marked active, the fair reading is that a substantial share of this collection is at **"built and demonstrated"** maturity rather than **"deployed and running unattended"** maturity. Several of the folder READMEs say as much. It should be said plainly and first, rather than discovered by an interviewer who thought to count.

This is not a criticism of the work. It is a criticism of any description of the work that implies 59 live production systems.

6 A taxonomy of the 59

Fifty-nine is too many to walk through one at a time, and they are not 59 unrelated things. They fall into seven families, and once you know the family you can predict most of the architecture.

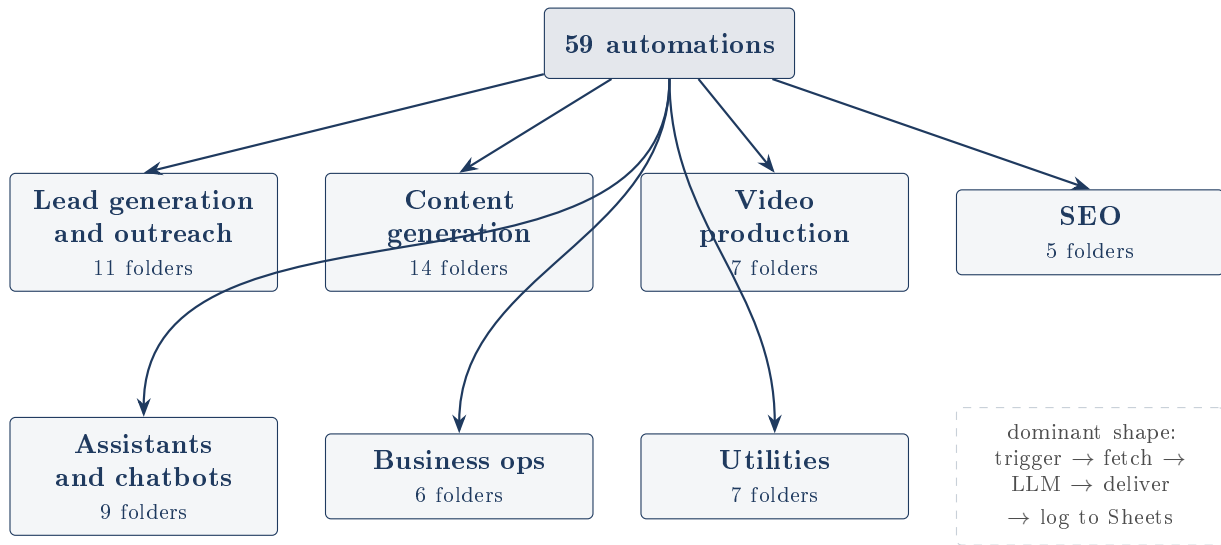


Figure 4: The seven families. Counts are of folders and sum to 59.

6.1 Family 1: Lead generation and outreach (11)

Find businesses, get contact details, write to them, chase them.

google-maps-to-email-scraper, linkedin-extractor-from-google-maps,
instagram-link-finder-outreach, cold-email-outreach-sequence-system,
deep-multiline-icebreaker-system, gmail-outreach-follow-ups, subscriber-email-sequence,
newsletter-automation, whatsapp-ai-sales-lead-system, upwork-ai-agent-pipeline,
instagram-viral-reel-scraper.

The shared shape: a scraping API supplies raw businesses, a cleaning step normalizes them, an LLM writes something personal per lead, a sending platform delivers it, and a spreadsheet records who has been contacted. This family contains the largest system here and Section 10 walks it.

6.2 Family 2: Content generation (14)

social-media-content-generator (190 nodes, the biggest single file), ai-social-media-amplifier,
social-media-poster, social-media-trend-monitor, instagram-posting-workflow,
linkedin-publish-utility, real-estate-market-update-poster, fb-ad-creative-generator,
text-to-image-generator, presentation-generator-tool, ai-proposal-generator-system,
daily-health-tip-bot, ai-motivational-quotes-bot, ai-workout-generator.

Shape: gather a fact or a prompt, ask an LLM to write copy, optionally ask an image model for a picture, publish or return it.

6.3 Family 3: Video production (7)

auto-video-generator, faceless-shorts-system, heygen-ai-video-production, submagic-video-clip-generator, instagram-reel-analyzer, instagram-reel-scraper-transcriber, youtube-agent-starter-template.

Shape: script $\rightarrow$ voice $\rightarrow$ render $\rightarrow$ poll $\rightarrow$ publish. This family is defined by the polling problem: video rendering takes minutes, so these workflows must ask "is it done yet?" on a loop, which is why 33 wait nodes exist in this repository.

6.4 Family 4: SEO (5)

seo-ai-agent, seo-keyword-research-tool, cluster-seo-post-generator, trend-seo-post-generator, competitor-content-gap-analyzer.

Shape: a keyword data API supplies volumes, an LLM clusters and writes, WordPress receives a draft. Notably these publish as *drafts*, never live, which is the right call and is stated as deliberate.

6.5 Family 5: Assistants and chatbots (9)

ultimate-personal-assistant-system, rag-ingestion-pipeline, tutorial-rag-chat-agent, content-business-chat-agent, telegram-joke-agent, telegram-translator-bot, telegram-checklist-generator, telegram-to-vapi-trigger, deep-research-report-generator.

Shape: a chat or messaging trigger, an agent node, a memory node, and a set of tools. Sections 8 and 9 walk the two most interesting.

6.6 Family 6: Business operations (6)

crm-lead-to-stripe-invoice, calcom-trigger-workflow, fathom-native-node-integration, expense-tracker-to-sheets, daily-email-digest-ai-powered, birthday-greetings-automation.

Shape: a real business event happens, and something is recorded, billed or announced. This is the family where a bug costs actual money, and Section 7 walks the one that bills people.

6.7 Family 7: Utilities (7)

uuid-generator-utility, website-screenshot-utility, youtube-transcript-fetcher, handwriting-to-text-ocr, booking-time-utility-set, vapi-call-trigger-utility, hacker-news-daily-digest.

Small, single-purpose, mostly webhook-in / answer-out. These are the building blocks other workflows call, and the smallest is 3 nodes.

7 Walkthrough 1: CRM lead to Stripe invoice

This is the automation to understand first. It is only 6 nodes, it is the example from Section 2, and it touches real money, which makes its flaws the most instructive in the repository.

7.1 What it does

When a sales task in ClickUp (a project management tool used here as a CRM) is moved to the closed-won status, a ClickUp automation posts to this workflow’s webhook. The workflow then fetches the full task, creates a Stripe customer, and issues a finalized invoice.

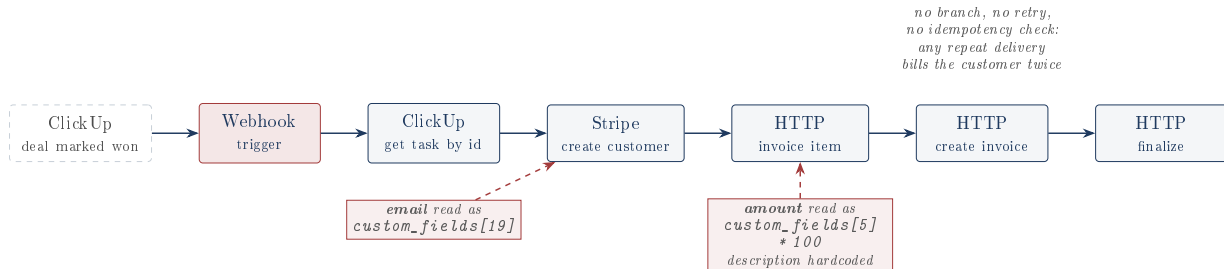


Figure 5: Six nodes, one straight line, no error path. The two red annotations are verified readings of the actual node parameters, not hypotheticals.

7.2 Why three raw HTTP calls sit next to a native Stripe node

The workflow uses n8n’s built-in Stripe node to create the customer, and then three hand-written HTTP calls to do the invoicing. That looks inconsistent, and it is worth explaining because it is the single most common shape in this repository.

n8n’s Stripe node supports creating customers. It does not support the three-step invoice dance Stripe requires (create an invoice *item*, create the *invoice*, then *finalize* it to actually send it). So the author used the native node where it fit, and dropped to raw API calls where it did not, reusing the same stored Stripe credential for both via n8n’s `predefinedCredentialType` option so the API key still never appears in the workflow.

This is the defining move of the whole collection

128 `httpRequest` nodes exist for this reason. The visual tool covers the common 80%; the remaining 20% is done by reading the vendor’s API docs and constructing the call by hand. Someone who can only drag boxes stops at the edge of what the boxes do. Being able to say why each of those 128 exists is what makes this a portfolio project rather than a folder of exports.

7.3 The flaws, verified

Every one of the following was confirmed by reading the node parameters directly, not inferred from the README.

Custom fields are read by array position

The Stripe node’s email is the expression `custom_fields[19].value`. The amount is `custom_fields[5].value * 100`.

ClickUp returns custom fields as an ordered array. Position 19 is the email *today*. Add a field to that ClickUp list, reorder the columns, or delete one, and 19 silently becomes something else. There is no name lookup, no assertion, no guard. The workflow does not fail; it invoices the wrong data to the wrong address, quietly, and the first person to notice is the customer. Fixing it is a small code node that finds the field by its name or ID. The fact that the README identifies this flaw itself is genuinely to the author’s credit; it is still present in the file.

No idempotency, on a node that charges people

The chain has no check for an existing Stripe customer with that email, and no check for an existing invoice for that deal. Webhooks are retried by their senders as a matter of routine when a response is slow. Two deliveries of the same event produce two customers and two invoices.

This is the highest-severity issue in the repository. Everything else here produces a bad tweet or a wasted API call; this one double-bills a real person. The standard fix is an idempotency key, which Stripe explicitly supports on exactly these endpoints.

The invoice line always says the same thing

The invoice item's description is the literal string "1st Deposit", not an expression. Every invoice this workflow has ever created carries that line, whatever the deal actually was.

8 Walkthrough 2: the Ultimate Personal Assistant

Five workflow files, 50 nodes. This is the collection's cleanest architecture and the best demonstration of the sub-workflow-as-tool pattern.

8.1 What it does

You text a Telegram bot, by typing or by voice. An orchestrating agent works out what you want and delegates to one of four specialist agents: email, calendar, contacts, or content creation. The answer comes back as a Telegram reply.

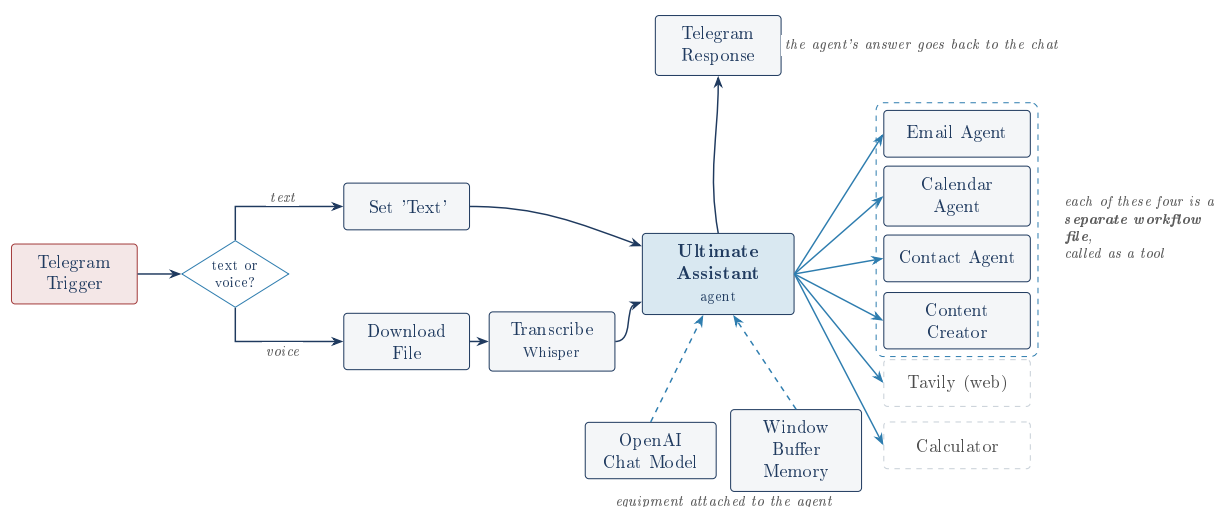


Figure 6: The orchestrator. Solid blue arrows from the agent are tool *calls* (the agent decides). Dashed arrows are equipment attachments. Note that the file stores the tool arrows pointing the other way; this diagram shows behaviour, not storage.

8.2 Why this design is good

Sub-workflow as tool is the automation equivalent of a function call

Each specialist is its own file with its own `executeWorkflowTrigger`. That buys exactly what functions buy in code:

- **Testability.** The Calendar Agent can be executed on its own with pinned input, without

going near Telegram.

- **Isolation.** Changing how contacts are looked up cannot break email.
- **Prompt size.** A single agent holding calendar, email, contacts and content instructions would carry an enormous system prompt and choose badly. Four focused agents each hold a small one.
- **Reuse.** The Email Agent is callable by anything else.

The alternative, which several other workflows in this repository chose, is one giant canvas. Compare the 190-node single file in `social-media-content-generator`.

The voice handling deserves a note: the Switch on the trigger's output distinguishes a text message from a voice note, and the voice branch downloads the audio and pushes it through a transcription node before rejoining the same agent. Both paths converge, so the agent never knows or cares which one it was. That is the correct place to put that seam.

The orchestrator is 15 nodes; the intelligence is in prompts you cannot see

Structurally this file is nearly empty: a trigger, a switch, two branches, an agent, six tools, a reply. Everything that decides *behaviour* lives in the agent's system prompt and in the four sub-agents' prompts.

That is not a flaw in the design, but it is a limit on what this document can tell you. Reading the graph proves the wiring is sound. It cannot prove the assistant is any good, and no amount of static reading will. That would need execution logs, which are not in the repository.

9 Walkthrough 3: the RAG ingestion pipeline

47 nodes. The most technically dense single automation here, and the one that best rewards a diagram.

9.1 What it does

Watch a Google Drive folder. When a document appears, work out its type, extract its text, chop it up, turn the chunks into embeddings, and store them in a Supabase vector table. Separately, a chat agent answers questions using that table as its source.

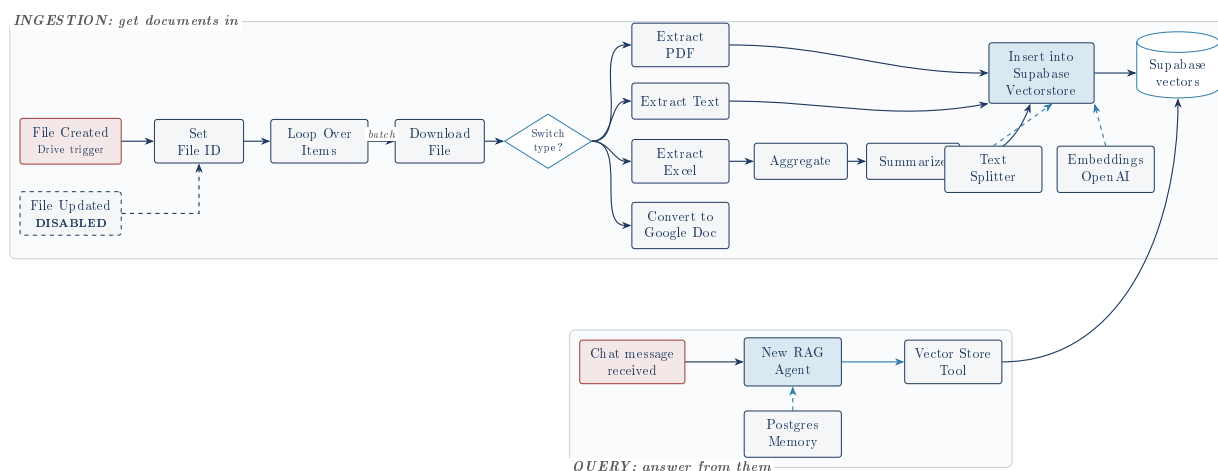


Figure 7: Two independent lobes sharing one vector table. The dashed "File Updated" path is a disabled trigger feeding a duplicate copy of the entire ingestion branch.

9.2 The interesting parts

The type switch. Documents arrive as PDFs, plain text, spreadsheets and Google Docs, and each needs different extraction. The Switch node routes six ways: three go to dedicated extractors, and the other three all converge on "convert to a Google Doc first, then handle it". That is a nice reduction: rather than write four more extractors, push the odd formats through a format you already handle.

The spreadsheet detour. Excel files do not go straight to the vector store; they go through Aggregate and then Summarize first. This is correct and shows real understanding of RAG. Embedding a spreadsheet row by row produces chunks like "Q3 | 41 | 12", which carry no retrievable meaning. Summarizing into prose first gives the embedding something to work with.

The delete-before-insert. On the update path, `Delete Old Doc Rows` runs before re-inserting. Without it, editing a document would leave the old chunks in the table alongside the new ones, and the agent would answer from both, confidently citing a version that no longer exists. Someone thought about this.

The entire ingestion branch exists twice, and the second copy is switched off

The graph contains two complete, parallel ingestion pipelines. The `File Created` trigger drives one (Set File ID1, Loop Over Items, Switch, Extract PDF Text1, Extract from Text File1, Extract from Excell1, Aggregate, Summarize, Insert into Supabase Vectorstore). The `File Updated` trigger drives a second, near-identical copy with different node names (Set File ID, Loop Over Items1, Switch2, Extract PDF Text, ..., Insert into Supabase Vectorstore1). The `File Updated` trigger is **disabled**.

So: roughly half of this 47-node workflow is a duplicate of the other half, and it is currently dead. Every future change to the extraction logic has to be made twice, in two places, with confusingly similar node names, and a mistake in the disabled copy will not show up until someone re-enables it months later.

The clean fix exists and is not exotic: both triggers feed a single shared branch, with the delete step gated behind a check for which trigger fired. This is copy-paste architecture, and it is the most expensive kind of debt in a visual tool, because the tool makes copying a canvas trivially easy and offers nothing that would make you factor it out.

10 Walkthrough 4: the cold email outreach system

Ten files, 275 nodes. The largest system here by a wide margin, and the one that best shows both the ambition and the structural problems of building at this scale in a visual tool.

10.1 What it does

Take a raw lead (name, company, LinkedIn URL). Research them. Write a six-touch email sequence personalized to what the research found. Push the finished sequence into Instantly.ai, a cold-email sending platform.

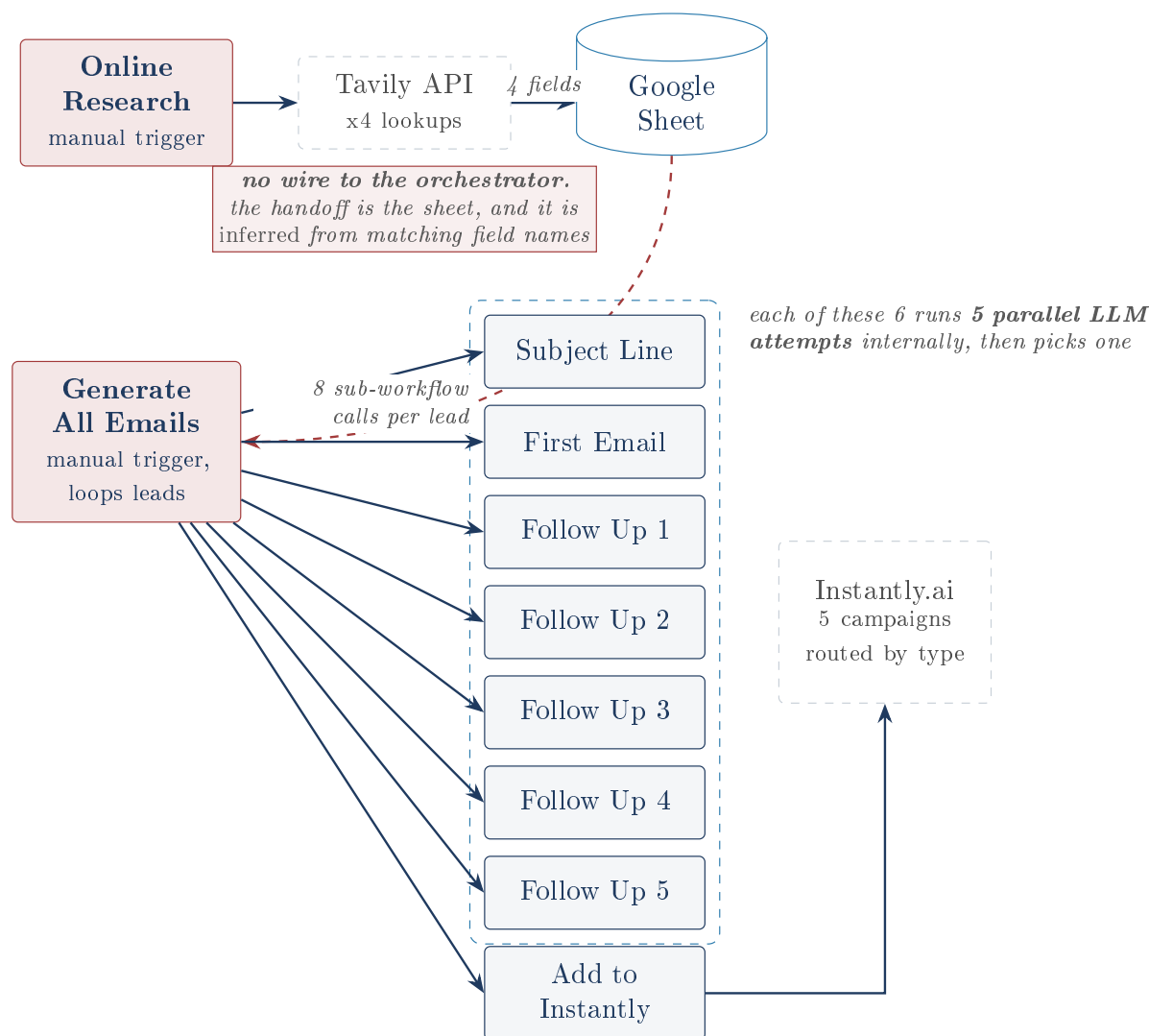


Figure 8: The cold email system. Up to 30 Gemini calls per lead before a single sequence is finished.

10.2 The five-attempt pattern

Each of the six copywriting workflows generates the same email five times in parallel (`generate_v1` through `generate_v5`), each with its own model node, then uses a switch/if gate to pick the first non-empty result.

Why generate the same thing five times?

Because an LLM call can come back empty, refuse, or return unusable formatting, and it does so unpredictably. Running one attempt and retrying on failure costs you the latency of a second round trip. Running five in parallel and taking the first good one costs you five times the money and none of the time.

It is a legitimate engineering trade, and it is the right one when a human is waiting. It is the wrong one when nobody is waiting, which is this case: this is a batch job over a lead list. The five-way fan-out buys latency that nobody is spending, at 5x the token cost. A single call with a retry-on-empty would be materially cheaper for the same result.

Six copies of the same logic, kept in sync by hand

The five-attempt block and its output-cleaning code node are duplicated across all six copy-writing workflows. There is no shared sub-workflow for the common part, even though this system *already uses* sub-workflows and therefore had the mechanism sitting right there.

A fix to the cleaning regex has to be made six times. The consequences of that are already visible in the repository: `NEW___Generate_All_Emails.json` still contains **nine orphaned `executeWorkflow` nodes** from an earlier version, disconnected from everything, left behind when the workflow was restructured. Verified: they have no incoming and no outgoing connections.

The research step is not actually connected to anything

`Online_Research.json` has a manual trigger and is called by no `executeWorkflow` node in any of the other nine files. Its four output fields (`general_search`, `news_search`, `person_linkedin_scrape`, `company_linkedin_scrape`) are read by name inside the prompts of every email workflow.

So the handoff is: run research by hand, let it write to the sheet, then run the orchestrator by hand, which reads the sheet. Two manual steps and an undocumented ordering constraint, with nothing in the system enforcing that research ran first. If someone runs the orchestrator on a fresh lead, the prompts get empty research fields and the LLM writes generic copy, with no error anywhere.

This handoff is inferred, from the matching field names, not confirmed by any node connection. There is no wire. The project's own README states this inference explicitly and honestly, and this reading independently confirms it.

No deduplication of leads, anywhere

Nothing in the ten files checks whether a lead was already researched, already written to, or already pushed to Instantly. Running the orchestrator twice on the same list generates and uploads the sequence twice. For a cold email system, whose entire premise is not annoying strangers, that is a consequential gap.

11 Cross-cutting patterns

Having read all 80 files, these are the habits that recur. They are more useful than any single walkthrough, because they are what the author would actually be asked to defend.

11.1 Google Sheets as the database

124 Google Sheets nodes and 5 Sheets triggers. Sheets is used here as: the lead database, the job queue, the state store, the log, and the human review interface.

This is smarter than it first looks

The instinct is to call this amateurish. Resist it for a moment, because a spreadsheet buys four things a real database does not:

- **The client can see it.** No admin panel to build.
- **The client can edit it.** Human-in-the-loop review, for free.
- **Zero provisioning.** No migrations, no hosting, no connection strings.
- **It is already the incumbent.** The work was in a spreadsheet before the automation

existed. Meeting the business where it is beats a migration.

For a small agency automating its own chores, this is a defensible engineering decision, not a shortcut. Saying so, and then saying where it breaks, is a much stronger answer than either defending it blindly or apologizing for it.

Where Sheets-as-database breaks, concretely

- **No transactions.** A workflow that reads a row, decides, and writes back has a race window. Two concurrent runs can both read "not yet emailed" and both email.
- **No unique constraint.** This is *why* the dedup problems in Section 10 exist. A database would have refused the duplicate.
- **Rate limits.** The Sheets API caps requests per minute per user. A batch job hits it.
- **It does not scale.** `cluster-seo-post-generator` reads the entire post history on every run to do internal linking. Fine at 50 rows. At 5,000, that is a very large prompt and a very slow run.

The honest position: right call at this size, with a known and stated ceiling.

11.2 Polling for slow jobs

33 `wait` nodes exist because image and video generation take minutes, and the APIs are asynchronous: you submit a job, get a ticket, and must ask later whether it is done.

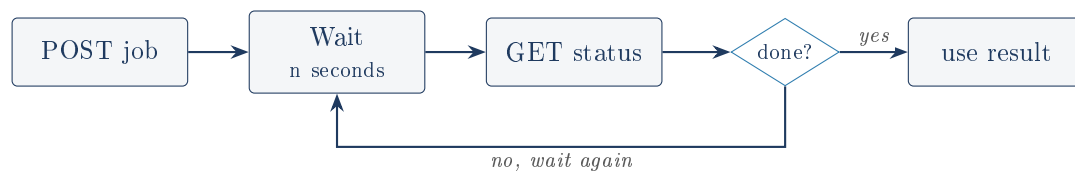


Figure 9: The polling loop, used across the video family.

None of these loops has a maximum attempt count

The polling loops here check "is it done?" and, if not, wait and check again. What none of them do is count. If a render job fails in a way that leaves it permanently not-done, the loop spins until `n8n`'s own execution timeout kills it, burning API calls the whole way. A bounded loop with a failure branch is the standard shape, and it is absent.

11.3 Trusting the LLM's output shape

The single most common bug class in this repository. The pattern is always the same: a prompt says "return only JSON" or "return only the quote", and nothing downstream checks that it did.

`n8n` has a structured output parser node, which forces the model's response into a declared schema and fails loudly when it does not fit. It appears **3 times in 1,645 nodes**, in just 2 of the 80 files (twice in `deep-research-report-generator`, once in `youtube-agent-starter-template`). In the other 78 files, output shape is a matter of hoping.

The verified worst case

`competitor-content-gap-analyzer`'s Clean Content node has this as its stored JavaScript source. Read the letters carefully:

```
1 if ($input.item.json.body){nnnn$input.item.json.content =
```

```
2 $input.item.json.body.replaceAll('/^s+|s+$/g', '')...
```

The newlines that should separate the statements have been collapsed into literal letter `n` characters, glued onto the next token. So the parser sees a variable called `nnnn$input`, which does not exist, and throws a `ReferenceError` the instant the `if` branch is taken. Separately, the two regular expressions are passed as *quoted strings* (`'/^s+|s+$/g'`) rather than as regex literals, so even with the newlines restored they would match nothing.

The node is dead. It has never cleaned anything.

And here is the part that matters: the node is configured with `continueOnFail: true` and `alwaysOutputData: true`. So the error is swallowed whole. The item passes through unmodified, `content` and `contentShort` are never set, the downstream `InfraNodus` call receives effectively nothing, and *the workflow reports success*. Every run of this automation has produced a content gap analysis based on no content.

The lesson is not "someone mangled a code node". It is that `continueOnFail: true` turned a loud crash into a silent wrong answer. Error-tolerance flags are set on 27 nodes in this repository, and each one is a place where a failure becomes invisible.

11.4 Sticky notes as documentation

186 sticky notes, more than any other node type. Someone genuinely documented these canvases as they built them.

Documentation drifts, and here it demonstrably has

In `calcom-trigger-workflow`, the sticky notes label two reminder stages "5-Minute Alert" and "Call Started Notification". The actual `If` conditions and the email copy in those same nodes put one at roughly one hour before the call and the other in the final ten minutes. A reader who trusts the sticky notes gets the timing wrong. Canvas comments rot exactly like code comments, and there is nothing to check them.

11.5 Orphaned nodes: the visual tool’s characteristic debt

An orphan is a node with no path from any trigger. It cannot ever run. In text code an unreachable function is at least invisible; on a canvas, an orphan sits there looking exactly like a working step. Twelve of the 80 files contain orphans. Verified by computing reachability from every trigger:

File	Orphans	What is stranded
whatsapp-ai-sales-lead-system	13	An entire agent, model, memory and vector store
cold-email...Generate_All_Emails	9	Old copies of the sub-workflow calls
social-media-trend-monitor	8	Four scraper agents and their merge
vapi-call-trigger-utility	8	An agent, a model, a loop
ai-proposal-generator-system	4	The whole Google Slides output path
faceless-shorts-system	4	An alternative video provider branch
booking-time-utility-set (x3)	3–4	A Calendly path, in all three files
presentation-generator-tool	2	Two of the three slide generators
social-media-poster	2	The YouTube Shorts path
fb-ad-creative-generator	1	A folder-creation step

The proposal generator is the clearest case

ai-proposal-generator-system has two form triggers. One is wired to the PandaDoc path and works. The other, feeding the Google Slides path, has this in the connections map:

```
1 "On form submission": { "main": [ [ ] ] }
```

An empty target list. The key exists, the wire does not. Four nodes (Replace Text, OpenAI, Google Drive, Gmail) are unreachable from any trigger.

That is worth dwelling on, because it is a trap for anyone auditing these files: a naive check of "is this trigger in the connections map?" answers *yes* and is *wrong*. You have to look at whether the list is empty. This document's reachability analysis was corrected after making exactly that mistake.

And in that stranded path sits a node that always writes the literal cost \$1,850 into the deck, regardless of what the form said, plus two Gmail nodes whose subject and body are hardcoded for one specific past client ("Re: Proposal for LeftClick", "Hey Nick"). If that trigger were ever reconnected without reading the nodes, it would send every client the same price and the same name.

12 Security posture

An n8n export is a notorious place to leak a secret. This repository was audited for that specifically, and the result is good enough to state precisely.

12.1 How n8n stores credentials, and why that saves this project

A node does not contain its API key. It contains a *pointer* to an entry in n8n's encrypted credential store:

```
1 "credentials": {
2   "stripeApi": {
3     "id": "hR3xK9...",      <-- which entry in the credential store
4     "name": "Stripe account" <-- its display label
5   }
6 }
```

Listing 4: The credential reference on a node. This is the whole of it. There is no field here that could hold a secret.

Verified across all 80 files: there are **373 credential references**, spanning **27 distinct credential types**, and every single one contains exactly two keys, **id** and **name**. Not one carries a value.

Why this matters more than it sounds

This is a property of n8n's design, not of the author's diligence, and it is the reason exporting workflows is safe at all. The secrets never enter the JSON, so they cannot leak through it.

What the author *is* responsible for is everything n8n does not protect: keys typed into an HTTP node's header field by hand, endpoints, chat IDs, spreadsheet IDs, email addresses. That is where an export normally leaks, and that is what the audit below checked.

12.2 The audit, and its result

Checked for	Result
Provider key formats	Scanned all 268 tracked files for OpenAI, Google, Slack, GitHub, Telegram, Apify and JWT key shapes. Three hits, all placeholders (Bearer YOUR_API_KEY_HERE and similar). No live key.
Credential values	373 references, all id/name only. Clean.
Bearer tokens	Six distinct literals, every one a placeholder. Clean.
Private endpoints	Every host referenced is a public vendor API (Google, Stripe, Tavily, Apify, Runware, Novita, Cal.com, ...). No Supabase project URL, no n8n instance hostname. Clean.
Email addresses	All placeholders (@example.com) except the owner's own business address. Clean.
Owner identity	The shared block, which normally carries the n8n account name and email, is scrubbed in all 30 files that have one. Clean.

Conclusion: no live credential is committed to this repository. That is a real result, and it was not free: n8n protects the credential store, but the author had to find and scrub the hardcoded keys, the account names and the emails by hand. The project README states that live keys were found and stripped, and this audit finds no survivor.

The scrub was done by hand, and it shows

The same field, the n8n account name inside the **shared** block, has been replaced with **sixteen different placeholder spellings** across the files: YOUR_EMAIL_HERE, owner@example.com, Portfolio Owner, It Expert, YOUR_N8N_PROJECT_NAME, REDACTED_USER, Project Owner, and ten more variants.

The security outcome is correct: nothing leaked. But sixteen spellings of one substitution is the signature of a manual pass, and a manual pass is one that can miss a file. A scripted scrub with a single canonical placeholder would be both tidier and genuinely safer, because it could be re-run over the repository to prove nothing was missed. As it stands, the proof is this audit, which is a snapshot.

Webhook paths are in the files, and that is a judgment call, not a bug

Each webhook node carries its **path**, a UUID. Combined with the n8n instance's hostname, that is the live URL of the endpoint. The hostname is not in these files, so the UUID alone opens nothing, and several of these webhooks have no authentication on them (**birthday-greetings-automation's** README says so plainly).

The risk is therefore low but not zero, and it rests entirely on the instance hostname staying private. Regenerating the paths on import would cost nothing and remove the question. This document does not reproduce any of them.

13 The honest summary

Everything in this section was verified against the exports. Nothing here is taken from the READMEs on trust, although the READMEs deserve real credit: they found most of it first.

13.1 What is genuinely good

- **The documentation discipline is exceptional.** 59 folders, 59 READMEs, 59 credential manifests, nine consistent headings, 9,304 lines of prose. Plus 186 sticky notes inside the canvases. This is far above the norm for automation work.
- **The self-criticism is real, and it is specific.** The *Challenges* sections are not marketing. They say "the content-cleaning code is broken", "the workflow always writes \$1,850", "custom fields are read by array index". Spot-checks against the JSON confirmed these claims exactly, including the subtle ones. A README that tells you where the body is buried is worth ten that do not.
- **No credentials leaked.** Section 12.
- **The sub-workflow architecture, where it is used, is correct.** The personal assistant and the Upwork pipeline are properly factored.
- **The hard parts were not dodged.** 128 hand-written API calls, an async polling pattern, a RAG pipeline with a considered chunking strategy and a delete-before-insert. This is not drag-and-drop work.

13.2 What is weak, ordered by what it would actually cost

1. **No idempotency on the invoicing workflow.** A retried webhook double-bills a real customer. Highest severity in the repository, and the only one that costs money on the first occurrence.
2. **Silent failures via `continueOnFail`.** 27 nodes. The `competitor-content-gap-analyzer` case proves the failure mode is not theoretical: a broken node produced wrong answers and reported success, indefinitely.
3. **Copy-paste architecture.** The duplicated RAG ingestion branch, the six duplicated copy-writing blocks, the nine orphaned call nodes. This is the characteristic debt of visual tools and it is present throughout.
4. **Orphaned nodes in 12 of 80 files.** 55-plus nodes that cannot run, indistinguishable on the canvas from ones that can.
5. **Index-based field access.** `custom_fields[19]` is a silent time bomb tied to someone else's column order.
6. **Unbounded polling loops.** No attempt cap anywhere in the video family.
7. **Hardcoded values where expressions belong.** \$1,850, "1st Deposit", "Hey Nick", a Telegram chat ID, the literal caption "hello" in `auto-video-generator`.
8. **No output validation on LLM responses.** The structured output parser node appears 3 times in 1,645 nodes, across just 2 of the 80 files.
9. **Documentation drift.** Sticky notes disagreeing with the nodes they label.

13.3 The maturity question, stated plainly

The one thing to get straight before any interview

28 manual triggers. 9 of 80 workflows active. 12 files with orphaned nodes. 45 disabled nodes.

This is a portfolio of **59 designed and built automations**, a meaningful fraction of which are at demonstration maturity rather than unattended-production maturity. The repository does not contain the evidence needed to claim otherwise, because execution logs are not in

it, and the `active` flags say what they say.

That is still a strong claim. "I designed and built 59 automations across seven problem domains, documented every one including its faults, and can tell you exactly which are production-hardened and which are not" is a better answer than "59 automations run my client's business", and it has the advantage of being defensible against someone who opens the files.

The failure mode to avoid is being caught claiming the stronger version.

13.4 What was inferred rather than confirmed

In keeping with the rule that inference must be labelled:

- **The taxonomy in Figure 4 is this document's own invention.** The repository does not group its folders. The seven families are a reading of what the automations do; the counts of folders are exact, the boundaries are a judgment call.
- **The research-to-orchestrator handoff in the cold email system is inferred** from matching field names. There is no connection in the JSON. The project's own README says the same and is equally explicit about it.
- **"Built and demonstrated rather than deployed" is an inference** from trigger types and active flags. It is a strong inference, but the alternative explanation, that workflows were routinely deactivated before export, cannot be ruled out from the files alone. The owner knows which is true; this document cannot.
- **Whether the disabled File Updated trigger in the RAG pipeline was switched off deliberately or abandoned mid-refactor cannot be determined** from the export. Both readings fit.
- **Whether the assistants actually work well is untestable here.** The graphs are sound; the behaviour lives in prompts, and prompts cannot be evaluated by reading them.

14 What an interviewer will ask

Five questions this repository invites, and the honest answer to each.

"Is this really engineering, or did you drag boxes?" 128 hand-written API calls where no node existed. An async polling pattern. A RAG pipeline that summarizes spreadsheets before embedding them, because embedding rows produces meaningless chunks. A sub-workflow architecture with a transcription seam. The boxes cover the easy 80%; the interesting part is the rest. And the failures in Section 13 are ordinary software failures, not tool failures, which is itself the argument.

"Why is a spreadsheet your database?" Because the client can see it and edit it, it needed no provisioning, and the work was already there. It breaks on concurrency, uniqueness and scale, and the duplicate-lead problem in the cold email system is exactly that bill coming due. Right call at that size, known ceiling.

"Your own README says a node is broken. Why is it still broken?" The honest answer is that finding it and fixing it are different jobs, and the repository's purpose was to document what exists rather than to repair it. The stronger version of the answer is that `continueOnFail` is what made it survivable long enough to become a documentation problem instead of an outage.

"Which of these actually run?" Nine are marked active. Say the number, then say which ones, then say why the rest are not. Do not let this one be discovered.

"What would you change first?" Idempotency on the invoice workflow, because it is the only one that costs money on the first mistake. Then delete `continueOnFail` from anything whose

silent failure produces a wrong answer rather than a skipped item. Then factor the duplicated RAG branch, because it is the change that gets more expensive every week it is not made.

All counts in this document were produced by parsing the 80 exported workflow files directly. Where the project's own documentation and the exports disagreed, the exports were treated as authoritative; no such disagreement was found on any claim spot-checked.